

← Go Back

Hardware

Portenta Max Carrier

Tutorials

Using NB-IoT or Cat-M1 with the Portenta Max Carrier

Connecting the Portenta Max Carrier to The Things Network

Cellular Connectivity with the Max Carrier & Pro 4G Module

Portenta Max Carrier User Manual

Getting Started with Portenta Max Carrier and Portenta X8

Home / Hardware / Portenta Max Carrier / Cellular Connectivity with the Max Carrier & Pro 4G Module

Cellular Connectivity with the Max Carrier & Pro 4G Module

Learn how to implement cellular connectivity using the mPCIe interface on the Portenta Max Carrier with the Arduino Pro 4G Module.

Author · Taddy Ho Chung

Last revision · 10/31/2024

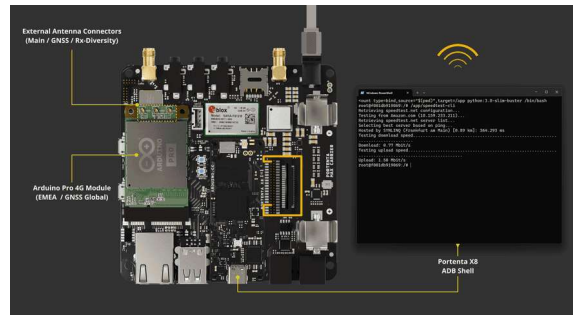
ON THIS PAGE

Overview

Goals	
Hardware and Software Requirements	+
Mini PCI Express	
Mini PCIe & Portenta Max Carrier	+
Pro 4G Module - Cat.4 Modem	
Complete Hardware Setup	
Instructions	+
Conclusion	
Support	+

Overview

This tutorial will guide you about the **Mini PCI Express (Mini PCIe) interface of the Portenta Max Carrier**. We will explore the onboard Mini PCIe slot and show you how to set up and configure this interface using the Portenta X8, ensuring you can fully leverage the capabilities of your device.



Portenta Max Carrier with the Portenta X8 & Pro 4G Module

The hands-on part of this tutorial will walk you through performing a speed test with the **Arduino Pro 4G Module**, a Cat.4 modem mini PCIe card compatible with the Portenta Max Carrier. This test aims to provide valuable insights into the functionality and efficiency of the Portenta Max Carrier and the Pro 4G Module while demonstrating its combination's practical application and network performance potential.

Goals

Learn about the onboard Mini PCI Express interface on the Portenta Max Carrier.

Learn to set up and configure the Mini PCIe interface on the Portenta Max Carrier using the Portenta X8.

Implement a speed test using a Pro 4G Module, the Portenta Max Carrier, and the Portenta X8.

Hardware and Software Requirements

Hardware Requirements

Portenta Max Carrier (x1)

Portenta X8 (x1)

Pro 4G Module (**GNSS Module Global / EMEA**) (x1)

Compatible antennas like the **Arduino Pro 4G Module Antennas Kit** (x1)

USB-C® cable (USB-C® to USB-A cable) (x1)

Wi-Fi® Access Point or Ethernet with Internet access (x1)

External antenna: Main antenna, GNSS antenna, and Rx-Diversity antenna

Power cables: Wires with a cross-sectional area ranging from 0.82 mm² to 1.3 mm², corresponding to AWG sizes 18 to 16

Software Requirements

To get your Portenta X8 ready for use with the Portenta Max Carrier, ensure it runs the most recent version of Linux. The steps to check and update your device are in [this section of Portenta X8's user manual](#).

Your Portenta X8 should be on Linux **image version 746 or newer** to ensure everything works smoothly. To update your  board to the latest image, use the **Portenta X8 Out-of-the-box** method or **manually flash it**, downloading the most recent version from this [link](#).

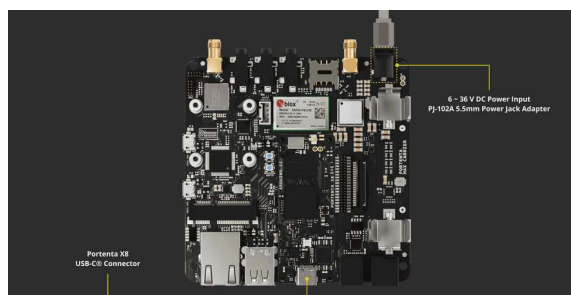
Setting Up the Hardware Power Distribution

For this tutorial, you will need to connect an **external 6.0 to 36.0 V power supply to the Portenta Max Carrier's Power Jack**. This ensures the carrier, the Portenta X8, and any PCIe modules connected are sufficiently powered.

To ensure the power demands are met, especially for the PMIC modules' external power, we recommend using cables that conform to the appropriate electrical standards, such as the ASTM B 258 standard and can carry currents up to 2.0 A. **Cables with a cross-sectional area ranging from 0.82 mm² to 1.3 mm², corresponding to AWG 18-16, should be adequate to manage 2.0 A of current.**

It is advised to use a **6.0 - 36.0 V external power source** when  using modules like the Arduino Pro 4G Module (EMEA / GNSS Global) or other mPCIe modules, to ensure a stable power supply for both the Portenta X8 and the carrier during prolonged usage.

The following image provides the position of the Power Jack on the Portenta Max Carrier:





Modems can be challenging to work with, so it is helpful to understand the software components involved to troubleshoot potential issues effectively.

Modems are complex devices, so it is recommended that the user knows each software component involved in order to troubleshoot potential issues.

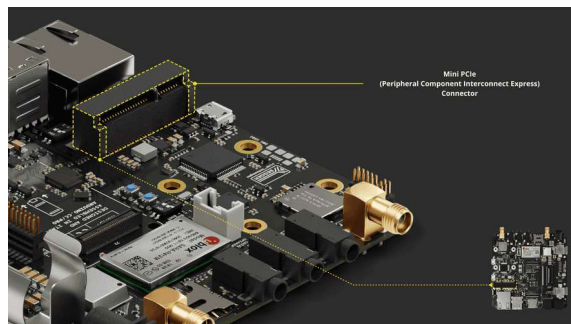
Mini PCI Express

Mini PCIe, short for Mini Peripheral Component Interconnect Express, is a smaller version of the PCIe interface mainly used in laptops and small devices to add features such as Wi-Fi®, Bluetooth®, and cellular connectivity.

These cards are significantly smaller than standard PCIe cards, typically measuring around 30 mm x 50.95 mm, and are designed to fit into the limited spaces of compact systems. They connect to a motherboard via a dedicated Mini PCIe slot, supporting PCIe and USB 2.0 interfaces. They are available in full-size and half-size variants.

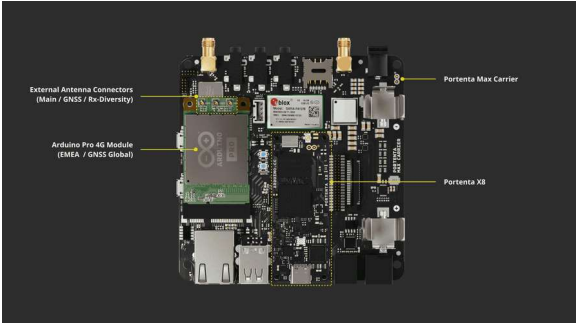
Mini PCIe & Portenta Max Carrier

The Portenta Max Carrier features a mini PCI Express card slot designed for use with female connectors. This slot is positioned at a right angle, and the board comes equipped with two detachable supports to ease the addition of external modules. The Max Carrier is also compatible with two sizes of Mini PCIe cards. Pins 8, 10, 12, and 14 are specifically set aside for use with SIM cards.



Mini PCIe Interface on the Portenta Max Carrier

In its portfolio, Arduino has two mini PCIe modules compatible with Portenta Max Carrier, the **Arduino Pro 4G Module**, a Cat.4 modem mini PCIe card available in two variants: **EMEA** and **GNSS Global**.



Portenta Max Carrier & PRO 4G Module

The onboard Mini PCIe slot of the Portenta Max Carrier has the following pin layout characteristics:

Pin Number	Silkscreen Pin	Power Net	Portenta Standard Pin	High-Density Pin	Pin Detail
2	N/A	+3V3 PCIE (Out)			From PCIE dedicated high current 3V3 power supply
4	N/A	Ground	GND	J1-22, J1-31, J1-42, J1-47, J1-54, J2-24, J2-33, J2-44, J2-57, J2-70	
7	N/A		PWM_9	J2-68	PCIE_CLKREQ
8	N/A				UIM_PWR
9	N/A	Ground	GND	J1-22, J1-31, J1-42, J1-47, J1-54 J2-24, J2-33, J2-44, J2-57, J2-70	
10	N/A				UIM_DATA
11	N/A		PCIE_CK_N	J2-19	
12	N/A				UIM_CLK
13	N/A		PCIE_CK_P	J2-17	

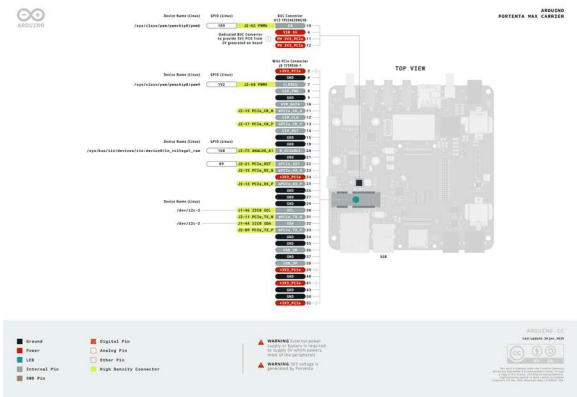
Pin Number	Silkscreen Pin	Power Net	Portenta Standard Pin	High-Density Pin	Pin Detail
15	N/A	Ground	GND	J1-22, J1-31, J1-42, J1-47, J1-54, J2-24, J2-33, J2-44, J2-57, J2-70	
18	N/A	Ground	GND	J1-22, J1-31, J1-42, J1-47, J1-54, J2-24, J2-33, J2-44, J2-57, J2-70	
20	N/A		ANALOG_A1	J2-75	W_DISABLE
21	N/A	Ground	GND	J1-22, J1-31, J1-42, J1-47, J1-54, J2-24, J2-33, J2-44, J2-57, J2-70	
22	N/A		PCIE_RST	J2-21	
23	N/A		PCIE_RX_N	J2-15	
24	N/A	+3V3 PCIE (Out)			From PCIE dedicated high current 3V3 power supply
25	N/A		PCIE_RX_P	J2-13	
26	N/A	Ground	GND	J1-22, J1-31, J1-42, J1-47, J1-54, J2-24,	

Pin Number	Silkscreen Pin	Power Net	Portenta Standard Pin	High-Density Pin	Pin Detail
				J2-57, J2-70	
27	N/A	Ground	GND	J1-22, J1-31, J1-42, J1-47, J1-54, J2-24, J2-33, J2-44, J2-57, J2-70	
29	N/A	Ground	GND	J1-22, J1-31, J1-42, J1-47, J1-54, J2-24, J2-33, J2-44, J2-57, J2-70	
31	N/A		PCIE_TX_N	J2-11	
33	N/A		PCIE_TX_P	J2-9	
34	N/A	Ground	GND	J1-22, J1-31, J1-42, J1-47, J1-54, J2-24, J2-33, J2-44, J2-57, J2-70	
35	N/A	Ground	GND	J1-22, J1-31, J1-42, J1-47, J1-54, J2-24, J2-33, J2-44, J2-57, J2-70	
36	N/A		USB_D_N		USB3_N (USB HUB)

Pin Number	Silkscreen Pin	Power Net	Portenta Standard Pin	High-Density Pin	Pin Detail
37	N/A	Ground	GND	J1-22, J1-31, J1-42, J1-47, J1-54, J2-24, J2-33, J2-44, J2-57, J2-70	
38	N/A		USB_D_P	J1-26	USB3_P (USB HUB)
39	N/A	+3V3 PCIE (Out)			From PCIE dedicated high current 3V3 power supply
40	N/A	Ground	GND	J1-22, J1-31, J1-42, J1-47, J1-54, J2-24, J2-33, J2-44, J2-57, J2-70	
41	N/A	+3V3 PCIE (Out)			From PCIE dedicated high current 3V3 power supply
43	N/A	Ground	GND	J1-22, J1-31, J1-42, J1-47, J1-54, J2-24, J2-33, J2-44, J2-57, J2-70	
50	N/A	Ground	GND	J1-22, J1-31, J1-42, J1-47,	

Pin Number	Silkscreen Pin	Power Net	Portenta Standard Pin	High-Density Pin	Pin Detail
				J2-33, J2-44, J2-57, J2-70	
52	N/A	+3V3 PCIE (Out)			From PCIE dedicated high current 3V3 power supply

The [full pinout document of the Portenta Max Carrier](#) can also provide a graphical representation of the mini PCIe interface pinout layout. The following image represents the dedicated page from the complete pinout documentation:



Portenta Max Carrier & PRO 4G GNSS Module Global

Mini PCIe Interface Setup

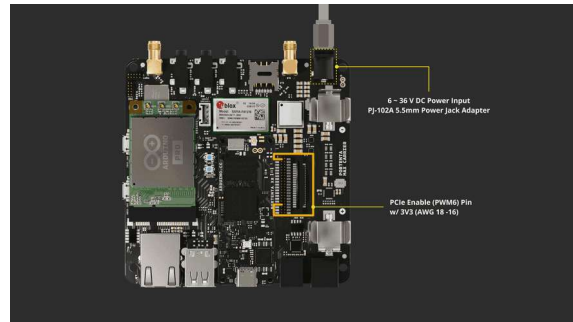
To address the power demands and ensure reliable connections, using jumper cables that comply with appropriate electrical standards, such as ASTM B 258 standard, and can support up to 2A of current is advisable. **Jumper cables with a cross-sectional area ranging from 0.82 mm² to 1.3 mm², roughly equivalent to AWG 18-16, should be sufficient for 2.0 A of current.**

Such a precaution is key to preventing wire overheating and guaranteeing stable power supply to Mini PCIe-compatible devices, like Cat.4 modems. An essential setup for activating the mini PCIe interface with the Portenta Max Carrier involves the following:

- Powering the **PCIE ENABLE (PWM6)** pin with 3.3 V
- Correctly installing a mini PCIe module, such as a Pro 4G GNSS Module Global or a Pro 4G EMEA Module

The **PCIE ENABLE (PWM6)** is the 33rd pin of the High-Density Connector (HDC) on the Portenta Max Carrier (PC).

Using an external power source of 6.0 to 36.0 V is important when working with an Arduino Pro 4G Module (EMEA / GNSS Global) or any other mPCIe modules because of their high power consumption. This ensures a continuous and stable power feed to the Portenta X8, the carrier, and the involved mPCIe module, especially during extended use.



Portenta Max Carrier & Mini PCIe Setup

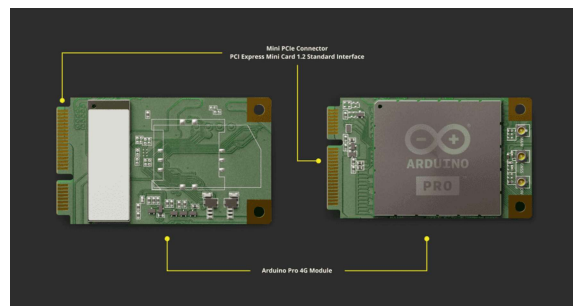
Pro 4G Module - Cat.4 Modem

The **Cat.4 modem**, compatible with **mPCIe interfaces**, leverages LTE (Long Term Evolution) standards to provide high-speed data transmission across various electronic devices.

These modems offer substantial data speeds, reaching up to 150 Mbps for downloads and 50 Mbps for uploads, making them ideal for a range of IoT and Industrial activities like remote monitoring, Smart Cities and buildings, smart farming and much more.

Designed with the mPCIe form factor, Cat.4 modems fit snugly in compact electronics such as laptops, tablets, and IoT devices with 4G connectivity. They also support older 3G and 2G networks, ensuring wide network coverage.

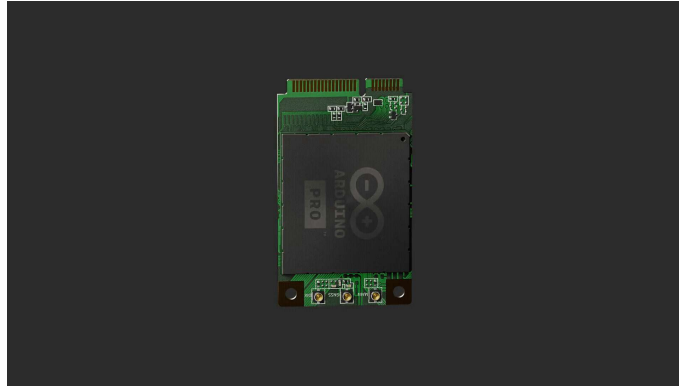
The Portenta Max Carrier uses this modem through its built-in mini PCIe interface. The **Arduino Pro 4G Module** is a key feature in this configuration.



Arduino PRO 4G GNSS Module

This module has two versions: **EMEA** and **Global (including the**

expansive smart city/building projects and to support remote maintenance and fleet management.

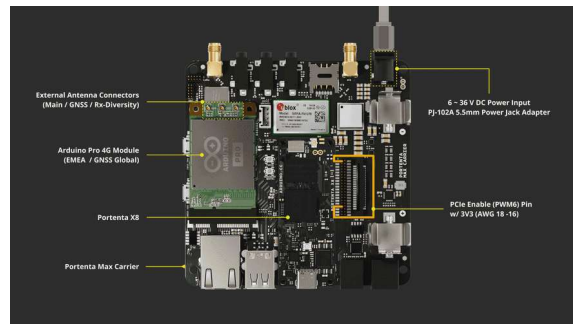


Make sure to attach external antennas to the Pro 4G Module to work correctly with wireless signals. There are three external

- i** antenna connectors: a main antenna connector, an Rx-diversity antenna connector, and a GNSS antenna connector. The impedance of the antenna connectors are 50 Ω .

Complete Hardware Setup

The image below illustrates the expected setup, showcasing the integration of the Portenta X8 and Pro 4G Module with the Portenta Max Carrier.



Portenta Max Carrier Mini PCIe & Portenta X8 Setup

Please ensure the mini PCIe power configuration is set as outlined in the [Mini PCIe Power Distribution section](#). **The Portenta X8 requires the PCIe ENABLE (PWM6) pin to be connected to a VCC (3V3) pin.**

Please use a 6.0 - 36.0 V external power source when using an Arduino Pro 4G Module (EMEA / GNSS Global) or any other mPCIe modules due to their high power consumption. This is important for maintaining a stable power supply to the Portenta X8, the carrier, and the involved mPCIe module, particularly for extended periods of use.



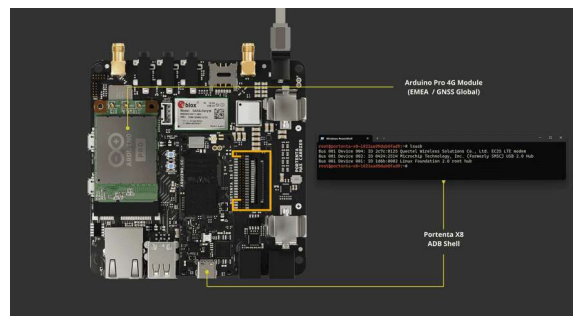
Accessing the Mini PCIe Interface

You can check if the mini PCIe module is properly installed and detected by the Portenta X8 by using the Portenta Max Carrier USB's functionality. Since the mini PCIe lanes on the Portenta Max Carrier include USB lines, and the Pro 4G Module functions as a USB device, it is possible to use them to verify a correct setup.

To ensure the Pro 4G Module is powered up and recognized by the Portenta X8 along with the Portenta Max Carrier, use the following command:

```
1 lsusb
```

This command lists the USB devices recognized by the Portenta X8. If the Pro 4G Module is correctly detected, you will see results similar to the ones shown in the following image.



Portenta Max Carrier Mini PCIe Module Listing

Setting Up Via Out-Of-The-Box Experience

Setting up the modem is easy with the Out-Of-The-Box process on the Portenta X8 paired with the Portenta Max Carrier.

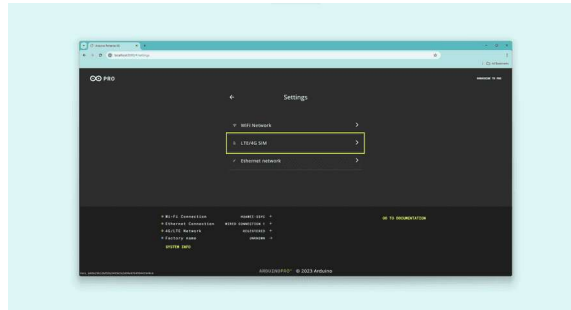
 If you are new to the Out-Of-The-Box experience of the Portenta X8, we recommend reviewing the [Out-Of-The-Box Experience section of the Portenta X8 User Manual](#) for a better understanding before you continue.

Proceed to the Out-Of-The-Box dashboard on the Portenta X8.



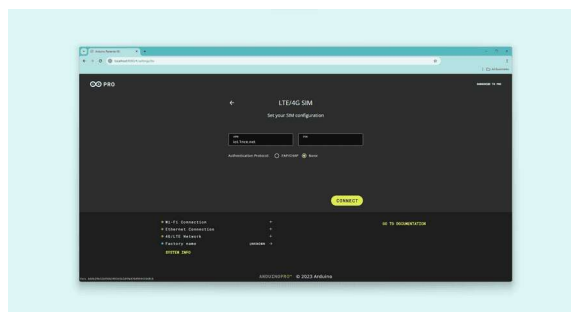
PRO 4G GNSS Module OOTB Activation - Main Page

In this dashboard, you will find the **Settings** option. Please click on this option to proceed to the next step.



PRO 4G GNSS Module OOTB Activation - Settings Option

In the **Settings**, choose the **LTE/4G Sim** option to start configuring the modem.



PRO 4G GNSS Module OOTB Activation - Modem Parameters

For the **LTE/4G Sim** settings, you will need to provide:

APN

PIN (if available)

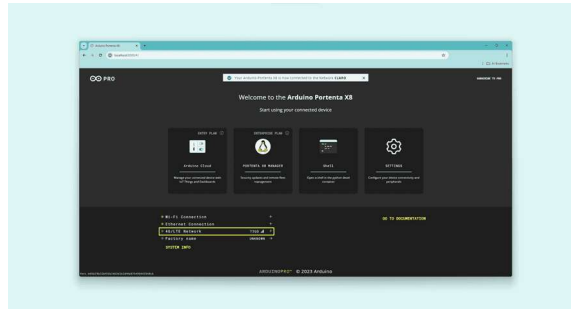
You will also need to choose an **Authentication Protocol**, either:

PAP/CHAP

NONE

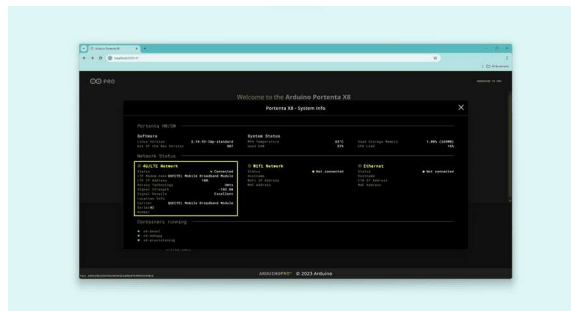
PAP (Password Authentication Protocol) sends passwords as clear text, suitable for low-security or legacy environments. At the same time, CHAP (Challenge-Handshake Authentication Protocol) offers enhanced security through a three-step verification process and encrypted passwords, providing better protection than PAP.

Enter these details based on your SIM card's requirements. Once done, the **4G/LTE Network** connection status will be displayed at the bottom left of the Out-Of-The-Box dashboard.



PRO 4G GNSS Module OOTB Activation - Network Status

A brief notification will confirm the successful network connection by the Portenta X8 if a network is available. You can view more details about the connection by selecting **SYSTEM INFO**.



PRO 4G GNSS Module OOTB Activation - System Information

You have successfully connected the Portenta X8 to a 4G/LTE network via the Portenta Max Carrier and the Pro 4G Module.

Setting Up Using Linux Environment

The Pro 4G Module can be managed via ADB shell on the Portenta X8's Linux environment. A series of commands is used to apply the necessary overlays for the Portenta X8 and the Portenta Max Carrier.

To check the existing device tree overlay configurations, which play a key role in managing hardware features and customizing the system, you can use this command:

```
1 fw_printenv overlays
```

This command uses the `fw_printenv` tool to list the overlays as per the U-Boot firmware settings. Overlays are crucial for specifying changes or additions to the device's hardware setup without modifying the original device tree structure.

This might involve enabling extra peripherals, setting up pin

thus offering a versatile means for customizing hardware settings. Among the overlays that should be visible is:

```
1 ov_carrier_max_pcie_mini
```

If it is not present, the following steps will help you set the needed overlays to use the mini PCIe interface on the Portenta Max Carrier. There are two ways to set up the overlay configuration for the mPCIe interface:

Via **tenta-config** based on the `tenta` framework

Via ADB shell

Overlay Configuration Via Tenta-Config

You can apply the necessary overlays to activate mini PCIe on the Portenta Max Carrier with the Portenta X8 by using the **tenta-config** process.

Start by accessing the docker container named **x8-devel** with this command:

```
1 docker exec -it x8-devel sh
```

This command uses **docker exec** to start a new shell session inside the active **x8-devel** container. The `-it` flag creates an interactive terminal session, allowing you to perform commands directly within the container's environment. This feature is convenient for development activities, such as modifying code, tracking processes, or troubleshooting within the container's secluded environment.

Find the **tenta_runner** Python® script with the help of this command:

```
1 find / -name *.py
```

This command begins at the root directory and recursively searches for files with a `.py` extension, representing Python® scripts. This search helps find Python®-based tools, scripts, or programs throughout the system.

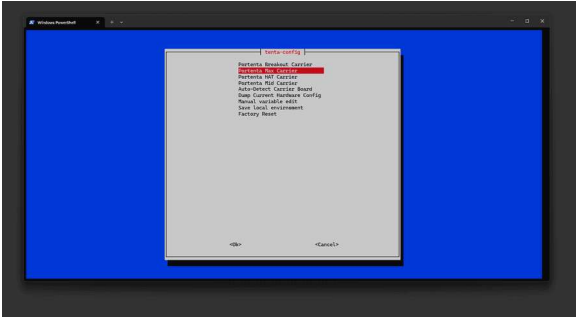
Once you have found **tenta_runner.py**, move to its directory with this command:

```
1 cd /root/examples/tenta-config
```

Run the **tenta_runner.py** script using the following command:

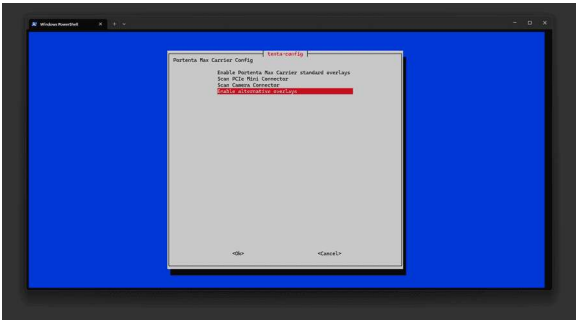
```
1 python tenta_runner.py
```

The script will activate a user interface within the `tenta` framework. Once the **tenta-config** window appears, please choose **Portenta Max Carrier**.



Portenta Max Carrier mPCIe Overlay Configuration - Main

It will then show a list of available options that can be executed within the Portenta Max Carrier platform. Here, the **Enable alternative overlays** option will be selected.



Portenta Max Carrier mPCIe Overlay Configuration - Overlays

It will open a new window displaying the **Adds Mini-PCIE support** option. Please select the **Adds Mini-PCIE support** option.



Portenta Max Carrier mPCIe Overlay Configuration -
Mini PCIe

It will prompt a message showing a new set of overlays that will be applied once modified.



Portenta Max Carrier mPCIe Overlay Configuration -
Applied Changes

Select **Ok** to confirm, and the device will be configured with the overlays for mini PCIe support.



Portenta Max Carrier mPCIe Overlay Configuration -
Change Set

Overlay Configuration Via ADB Shell

The following commands are used to set the required overlays to add mini PCIe support on the Portenta X8 and the Portenta Max Carrier.

```
1 fw_setenv is_on_carrier yes
```

```
1 fw_setenv carrier_name max
```

The previous commands are used to set environment variables, which we will use to set and apply the needed overlays to link the USB modem interface under its profile. The overlays are as follows and set using the following command:

```
1 fw_setenv overlays 'ov_som_lbee5k11dx ov_som_x8h7
```

NOTE: The `ov_carrier_max_pcie_mini` overlay is **not required for modems** such as the **GNSS Global (EG25)** and **EMEA (EC200A-EU)** variants of the Pro 4G Module, as these modems rely on the USB interface and not on the PCIe bus.

Once the overlays are set, please reboot the Portenta X8 to ensure the configuration has been applied correctly.

Connecting & Testing Network Connectivity

GNSS Global EG25 Module

The **GNSS Global (EG25) Module** supports the *Qualcomm MSM Interface (QMI)*, which is compatible with **NetworkManager**. You can configure the modem using **nmcli**, the command-line tool for **NetworkManager**:

```
1 nmcli c add type gsm ifname cdc-wdm0 con-name wwa
```

i For information on managing interface configuration rules for the EG25 module, please see [this section](#).

If your SIM card requires a PIN, update the command as follows:

For instance, if you are using Vodafone in Italy, you can replace the `<APN>` field with `mobile.vodafone.it` and include the PIN number as well:

```
1 nmcli c add type gsm ifname cdc-wdm0 con-name wwa
```

EMEA EC200A-EU Module

The **EMEA (EC200A-EU) Module** mainly uses raw AT commands over a USB serial interface and is not compatible with QMI. While it is not *natively supported* by **ModemManager**, basic connectivity can still be established using **mmcli**:

```
1 mmcli -m 0 --simple-connect='apn=iot.1nce.net,ip-
```

 The **EC200A-EU** modem is not compatible with **QMI**. It requires raw AT commands over a USB serial interface. For more information, refer to the [Quectel EC200A-EU documentation](#).

The latest images include the necessary `udev` rules to automatically manage the `ec200aeu` interface. You can verify this by checking the `75-ec200aeu.rules` file using the following command:

```
1 cat /etc/udev/rules.d/75-ec200aeu.rules
```

This rule file typically contains the following:

```
1 SUBSYSTEM=="net", ACTION=="add", ATTRS{idVendor}=
2 ACTION=="add", SUBSYSTEM=="net", KERNEL=="ec200ae
```

These rules automatically manage the `ec200aeu` network interface and ensure the required service starts.

For QMI Based Modems

QMI-based modems use the *Qualcomm MSM Interface (QMI)*, a messaging format for communication between software

services using either a **request/response** format or **unsolicited events** for system notifications.

To check if a modem supports QMI, use the following command:

```
1 qmicli -d /dev/cdc-wdm0 --dms-get-model
```

If the modem is QMI compatible, you can manage the **Raw IP mode** and control network connections using **qmicli**, which is an alternative to **ModemManager**. Before using **qmicli**, it is recommended to stop and disable the **ModemManager** service to avoid conflicts:

```
1 sudo systemctl stop ModemManager
```

```
1 sudo systemctl disable ModemManager
```

Once **ModemManager** has been disabled, you can use **qmicli** to communicate reliably with the QMI interface for modem operations.

 The **EC200A-EU** modem is not compatible with **QMI**. It requires raw AT commands over a USB serial interface. For more information, refer to the [Quectel EC200A-EU documentation](#).

Raw IP Mode Setup for QMI Based Modems

With the overlays configured, the setup process involves bringing down the `wwan0` interface, setting it to raw IP mode, and then bringing it back up:

```
1 ip link set dev wwan0 down
```

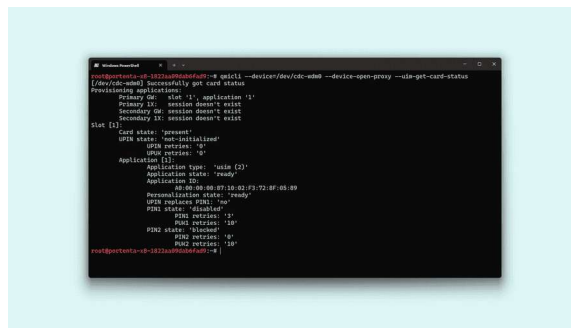
```
1 echo Y > /sys/class/net/wwan0/qmi/raw_ip
```

```
1 ip link set dev wwan0 up
```

Inspecting and Configuring the QMI Based Modem

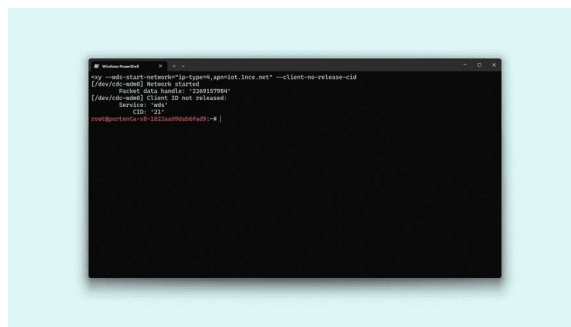
Following that, use `qmcli` commands to inspect the card's status and begin a network connection:

```
1 qmicli --device=/dev/cdc-wdm0 --device-open-proxy
```



PRO 4G GNSS Module - Card Status

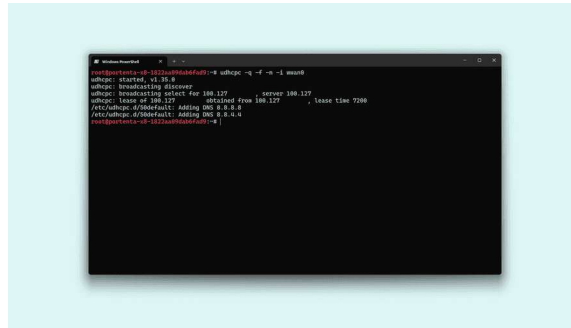
```
1 qmicli --device=/dev/cdc-wdm0 --device-open-proxy
```



PRO 4G GNSS Module - Network Initialization

After establishing the network connection, the `udhcpd` is used to handle dynamic IP configuration:

```
1 udhpcp -q -f -n -i wwan0
```



PRO 4G GNSS Module - Dynamic IP Configuration

Modem Power Management

Modems can become unresponsive, so it is recommended that power can be controlled through software to allow rebooting when necessary. By default, this process is handled automatically by **ModemManager** using customized scripts such as:

```
/usr/sbin/modem_on.sh
```

```
/usr/sbin/modem_off.sh
```

These scripts contain the logic to manage modem power for different carrier boards and are used by **ModemManager** to distinguish between each board type. You can review these scripts by checking the **ModemManager** service configuration:

```
1 systemctl cat ModemManager
```

If you prefer to manage the modem manually, you can use these scripts directly, as they provide a simpler way to handle modem power. For cases where **ModemManager** is disabled, you can use the **gpio** command to control the modem's power and add a 20 second delay for proper initialization:

```
1 gpio pin 5=1 #PCIE 3V3 BUCK EN (stm32h7
```

This is applicable to both USB based and QMI based modems. After powering on the modem, allow **20 seconds** for the modem to initialize properly:

```
1 sleep 20
```

Docker Container Considerations

In a Docker environment, it is often useful to disable **ModemManager** to avoid conflicts and instead control the modem using **qmicli** for QMI-based modems or use **nmcli/mmcli** for USB-based modems:

```
1 sudo systemctl stop ModemManager
```

```
1 sudo systemctl disable ModemManager
```

For modem power management, the connection between the **PCIE Enable (GPIO5)** pin and **VCC (3V3)** pin is required for a proper power setup. This bridge ensures proper hardware functionality when controlling the modem power. Moreover, you should manage power through software to allow the modem to be rebooted in case it becomes unresponsive.

Ensure that the Docker container has access to the GPIO device files by passing them into the container:

```
1 docker run --device /dev/gpiochip5 <docker-image>
```

The `<docker-image>` field is the name of the Docker image you want to run with access to the GPIO device files. For example, if your Docker image is called `my_modem_image`, the command would look like this:

```
1 docker run --device /dev/gpiochip5 my_modem_image
```

Inside the container, an **entrypoint.sh** script can control the modem's power via GPIO, with the 3.3V Buck Converter line connected to the **PCIE Enable (GPIO5)** pin. The following command can be added to the script:

```
1 gpioset gpiochip5 5=1
```



It is required to have **PCIE Enable (GPIO5)** pin connected to the **VCC (3V3)** pin to secure the power supply line.

This will enable the power to the modem and add a delay for proper modem initialization:

```
1 sleep 20
```

Testing Network Connectivity and Speed

We now have the Pro 4G Module with the Portenta X8 on the Portenta Max Carrier ready for use. To test the connection speed, perform a speed test by downloading the `speedtest-cli` script, making it executable, and running it within a Docker container:

```
1 wget -O speedtest-cli https://raw.githubusercontent.com
```

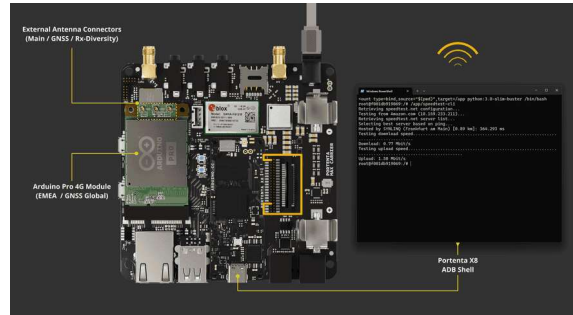
```
1 chmod +x speedtest-cli
```

```
1 docker run python:3.8-slim-buster
```

```
1 docker run -it --mount type=bind,source="$(pwd)",
```

```
1 /app/speedtest-cli
```

After the speed test, you might observe results similar to the following image.



Arduino Pro 4G Module - Speed Test



The download and upload speed may vary depending on the region.

For a more streamlined approach, you can use the following single-line command:

```
1 nmcli c add type gsm ifname cdc-wdm0 con-name wwa
```

If your SIM card requires a PIN, adjust the command as follows:

```
1 nmcli c add type gsm ifname cdc-wdm0 con-name wwa
```

For instance, if you are using Vodafone in Italy, you can replace the `<APN>` field with `mobile.vodafone.it` and include the PIN number as well:

```
1 nmcli c add type gsm ifname cdc-wdm0 con-name wwa
```

Conclusion

In this tutorial, you have learned the key steps to use and configure the mini PCI Express interface on the Portenta Max Carrier with the Portenta X8. You will be able to manage the mini PCIe interface and use the Pro 4G Module via the Out-Of-The-Box experience method, `tenta` framework, and ADB shell. You have seen exemplified performance and connectivity through a practical speed test using the Pro 4G Module, setting the stage for future projects and applications.

Support

If you encounter any issues or have questions while working with the Portenta Max Carrier, we provide various support resources to help you find answers and solutions.

Help Center

Explore our Help Center, which offers a comprehensive collection of articles and guides for the Portenta Max Carrier. The Arduino Help Center is designed to provide in-depth technical assistance and help you make the most of your device.

[Portenta Max Carrier help center page](#)

Forum

Join our community forum to connect with other Portenta Max Carrier users, share your experiences, and ask questions. The forum is an excellent place to learn from others, discuss issues, and discover new ideas and projects related to the Portenta Max Carrier.

[Portenta Max Carrier category in the Arduino Forum](#)

Contact Us

Please get in touch with our support team if you need personalized assistance or have questions not covered by the help and support resources described before. We're happy to help you with any issues or inquiries about the Portenta Max Carrier.

[Contact us page](#)

Suggest changes

The content on docs.arduino.cc is facilitated through a public [GitHub repository](#). If you see anything wrong, you can edit this page [here](#).

Need support?

[Help Center](#)
[Ask the Arduino Forum](#)
[Discover Arduino Discord](#)

License

The Arduino documentation is licensed under the [Creative Commons Attribution-Share Alike 4.0](#) license.

Was this article helpful?

